MCMP117 · Week 2 · Course Notes

Fundamentals of Operating Systems

Definitions, Goals, Components & Operating-System Structures

What exactly is an operating system, what is it for, and how is it built?

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. State a precise definition of an operating system and of the *kernel*.
2. Explain the two classic views of an OS: *extended machine* and *resource manager*.
3. Name the three design goals of an OS and explain why they conflict.
4. Describe the four components of a computer system, the role of interrupts, the storage hierarchy and dual-mode operation.
5. Compare the simple, monolithic, layered, microkernel, modular and hybrid OS structures, with one example of each.
6. Read and run small C programs that make system calls, and measure the cost of a system call.

Contents

1 What Exactly Is an Operating System? 1

1.1 Two classic views . . . 2

1.2 Kernel, system programs and application programs . . . 2

2 Design Goals of an Operating System 3

2.1 The goals conflict . . . 3

3 Components of a Computer System 4

3.1 Hardware organisation . . . 4

3.2 Interrupts . . . 5

3.3 The storage hierarchy . . . 5

3.4 Dual-mode operation . . . 6

4 Hands-on: Watching a Program Call the Kernel 6

4.1 Example 1: library call versus system call 7
4.2 Example 2: how expensive is a system call? 7

5 Operating-System Structures 9

5.1 Simple structure: MS-DOS 9
5.2 Monolithic kernel: UNIX and Linux 9
5.3 Layered approach 10
5.4 Microkernel 10
5.5 Modular kernel: loadable kernel modules 11
5.6 Hybrid systems 12
5.7 Comparison 13

6 Summary 13

7 Glossary 14

8 Try It Yourself 15

9 Review Questions 15

10 Looking Ahead: Week 3 16

1 What Exactly Is an Operating System?

Last week we used an informal definition: *the OS is the program that starts first and manages everything else*. Textbooks give more precise definitions, each looking at the OS from a different angle.

Angle	Definition
Position	A program that acts as an *intermediary* between the user of a computer and the computer hardware.
Permanence	The one program running at all times on the computer, usually called the **kernel**. Everything else is either a system program or an application program.
Function	Software that controls the execution of programs and manages the computer's resources: processor, memory, files and devices.

Working definition for this course

The **operating system** is the software layer that manages the hardware and offers programs a simpler, safer machine to run on. Its always-resident core is called the **kernel**.

1.1 Two classic views

Andrew Tanenbaum describes the operating system from two opposite directions.

Top-down: the OS as an extended machine. Real hardware is complicated and ugly. A disk is a set of magnetic platters or flash cells divided into sectors, with its own commands, error codes and timing rules. Almost no programmer wants to deal with that. The OS hides these details behind clean **abstractions**: you see *files* instead of sectors, *processes* instead of registers and interrupt lines, *windows* instead of video memory. Seen this way, the OS turns the real machine into a nicer, "extended" machine that is convenient to program.

Bottom-up: the OS as a resource manager. Many programs and users compete for one set of hardware. The OS decides who gets what, and when. It does this by **multiplexing** (sharing) resources in two ways:

- **Time multiplexing:** programs take turns. The CPU runs one program for a few milliseconds, then another; a printer prints one document, then the next.
- **Space multiplexing:** the resource is divided into pieces. Main memory holds several programs at once; a disk holds files belonging to many users.

Both views describe the same software. Most of this course (processes, CPU scheduling, memory, files) studies the OS as a resource manager.

1.2 Kernel, system programs and application programs

Not everything that ships with an operating system is the kernel.

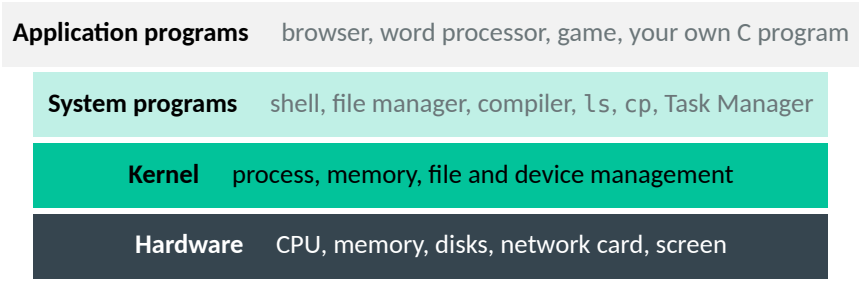


Figure 1: Software layers on a typical computer.

- The **kernel** is always in memory, runs in a privileged CPU mode (Section 3.4) and talks directly to the hardware.
- **System programs** ship with the OS but run as ordinary programs. They make the system usable.
- **Application programs** are everything you install to do your actual work.

Strictly speaking, "Linux" is only a kernel. A *Linux distribution* such as Ubuntu or Fedora adds thousands of system programs, most of them from the GNU Project.

> **Is the web browser part of the OS?**
> In 1998 the United States government sued Microsoft, partly over its claim that Internet Explorer was an

integral part of Windows. The courts found that Microsoft had illegally maintained its monopoly, and the question "what belongs to the OS?" became a legal question as well as a technical one. A useful rule of thumb: the kernel is certainly the OS; beyond the kernel, "the OS" is whatever the vendor ships as the standard system.

2 Design Goals of an Operating System

William Stallings summarises the objectives of an operating system in three words.

1. **Convenience.** The OS makes the computer easier to use and to program: files, windows, a shell, simple system calls. Users should not need to know the hardware.
2. **Efficiency.** The OS uses the hardware well: the CPU busy with useful work, memory full of useful data, devices working in parallel. The OS itself should have low **overhead**, i.e. consume few resources of its own.
3. **Ability to evolve.** New hardware (SSDs, GPUs, 5G modems) and new services must be addable without breaking existing programs. This goal, more than any other, drives the choice of OS *structure* (Section 5).

2.1 The goals conflict

The three goals pull in different directions, and different kinds of computers weight them differently.

Kind of system	Main priority	Why
Personal computer	Convenience	One user who wants things to "just work"; the CPU is idle most of the time anyway.
Server / cloud	Efficiency	Thousands of users; 1% more throughput can mean hundreds of fewer machines.
Smartphone	Efficiency (energy)	Battery life: the OS aggressively puts apps and the CPU to sleep.
Embedded / IoT	Efficiency (size, cost)	A washing-machine controller may have a few kilobytes of memory; there may be no OS at all.
Real-time (brakes, drones)	Predictability	An answer that arrives late is a wrong answer: deadlines must always be met.

Convenience versus efficiency

A program must read 10^6 numbers from a file. Suppose each system call (a request to the kernel, Section 4) costs $t_s \approx 0.1\ \mu$s of pure overhead.

- *Convenient:* ask the OS for one number at a time: 10^6 calls, overhead $10^6 \times 0.1\ \mu\text{s} = 0.1\ \text{s} = 100$ ms.
- *Efficient:* ask for blocks of 4096 bytes, about 10^3 numbers each: 10^3 calls, overhead $10^3 \times 0.1\ \mu\text{s} = 0.1$ ms.

The efficient version has 1000 times less overhead. The C standard library gives you both: you call `fgetc()` one character at a time, but the library reads large blocks behind the scenes. This technique is called **buffering**.

3 Components of a Computer System

A computer system can be divided into four components.

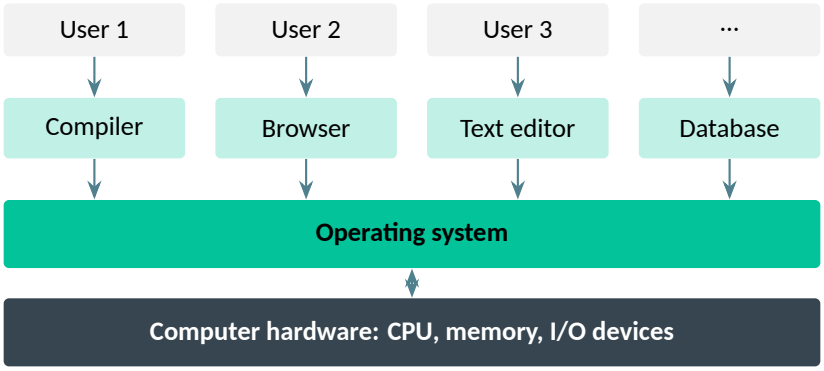


Figure 2: The four components of a computer system.

- **Hardware** provides the basic computing resources: CPU, memory, input/output (I/O) devices.
- **Application programs** use these resources to solve the users' problems.
- **Users** are people, but also other machines and programs (e.g. a web server's "users" are other computers).
- **The operating system** controls the hardware and coordinates its use among the applications and users.

3.1 Hardware organisation

A modern computer has one or more CPUs and several **device controllers**, all connected by a common **bus** to shared main memory.

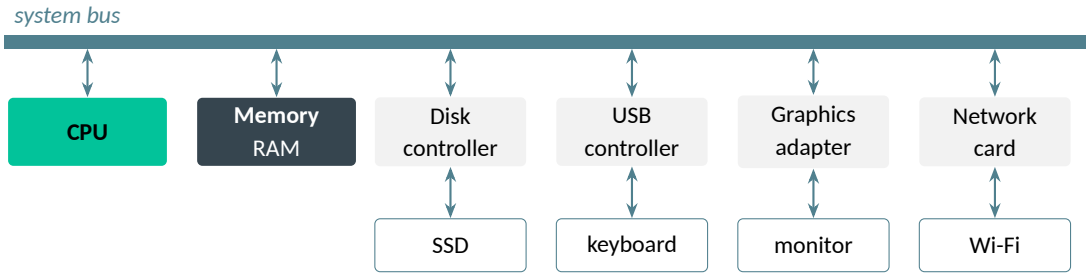


Figure 3: CPU, memory and device controllers share the system bus.

Each device controller has a small local buffer and works *in parallel* with the CPU. For each controller, the OS contains a **device driver**: a piece of software that understands the controller's commands and offers the rest of the OS a uniform interface.

3.2 Interrupts

How does the CPU find out that a device has finished its work? It could keep asking ("anything new?"), a technique called **polling**, but that wastes CPU time. Instead, the device *tells* the CPU by raising an **interrupt**:

1. The device controller finishes an operation (e.g. the data from the disk is now in its buffer).
2. It sends a signal on an interrupt line to the CPU.
3. The CPU saves the state of the program it was running and jumps to the **interrupt handler** whose address is stored in the **interrupt vector**, a table indexed by device number.
4. The handler (part of the device driver, inside the OS) processes the event.
5. The saved state is restored and the interrupted program continues as if nothing had happened.

Modern operating systems are **interrupt-driven**: if no program wants to run and no device needs attention, the OS simply waits. Every key press, mouse movement, network packet and finished disk read arrives as an interrupt, often thousands per second.

Why not just poll?

A keyboard produces at most about 10 key presses per second. Suppose checking the keyboard once takes $t_{\text{check}} = 1\ \mu\text{s}$ and, to feel responsive, we poll it $f = 1000$ times per second. The fraction of CPU time spent polling is

$$f \cdot t_{\text{check}} = 1000\ \text{s}^{-1} \times 10^{-6}\ \text{s} = 10^{-3} = 0.1\%.$$

That sounds small, but 99% of those checks find nothing, and a computer has dozens of devices. With interrupts, the CPU spends time on the keyboard only when a key is actually pressed, about 10 times per second.

3.3 The storage hierarchy

Storage comes in many types, which differ in speed, size, cost and *volatility* (whether the content survives power-off).

Level	Typical size	Access time	If a register access took 1 s...
CPU registers	< 1 KB	≈ 0.3 ns	1 second
Cache (L1–L3)	a few MB	1–40 ns	3 seconds to 2 minutes
Main memory (RAM)	8–64 GB	≈ 100 ns	≈ 5 minutes
Solid-state drive (SSD)	0.5–4 TB	≈ 50 μs	≈ 2 days
Hard disk drive (HDD)	1–20 TB	≈ 5 ms	≈ 6 months

The last column multiplies every access time by the same factor $1/(0.3\text{ ns}) \approx 3.3 \times 10^9$; for example, for a hard disk $5\text{ ms} \times 3.3 \times 10^9 \approx 1.7 \times 10^7$ s ≈ 190 days. Registers, cache and RAM are **volatile**; SSDs and disks are **non-volatile**. Much of the OS's work consists of moving data between levels so that the data a program needs is in fast storage when it needs it. We return to this in Weeks 6 and 7.

3.4 Dual-mode operation

If any program could do anything, one buggy or malicious program could destroy all the others. Protection therefore needs *hardware* support. The CPU has a **mode bit**:

- **Kernel mode** (also: supervisor, system or privileged mode; mode bit $= 0$): every instruction is allowed. Only the OS kernel runs in this mode.
- **User mode** (mode bit $= 1$): **privileged instructions**, such as direct I/O, changing memory protection or halting the CPU, are forbidden.

If a user program tries a privileged instruction, the hardware does not execute it; instead it *traps* to the OS, which usually terminates the program. When a program needs a privileged service, it makes a **system call**: a special instruction that switches to kernel mode and jumps to a fixed entry point in the kernel. When the kernel is done, it switches back to user mode and returns.

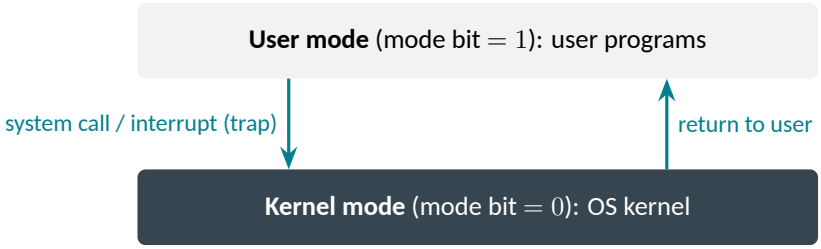


Figure 4: Switching between user mode and kernel mode.

On x86 processors the modes are called *rings*, numbered 0 (most privileged) to 3; Linux and Windows use only ring 0 (kernel) and ring 3 (user). The MS-DOS story in Section 5.1 shows what happens without this protection.

4 Hands-on: Watching a Program Call the Kernel

The programs in this section are in the `code/` folder next to these notes. They are written in C for Linux. You can compile them on the lab computers, in WSL (Windows Subsystem for Linux), or in an online C compiler.

You are *not* expected to write C in this course; read the comments and focus on what the program does.

4.1 Example 1: library call versus system call

```c
/* hello_syscall.c -- the same message printed in two ways.
 * Compile:  gcc hello_syscall.c -o hello_syscall
 * Run:      ./hello_syscall
 * Trace:    strace ./hello_syscall      (Linux only)
 */
#include <stdio.h>    /* printf: library function (user mode)   */
#include <unistd.h>   /* write:  thin wrapper around a system call */

int main(void)
{
    /* 1) Library call: printf formats the text in user mode,
     *    then asks the kernel to output it via write().      */
    printf("Hello from printf!\n");
    fflush(stdout);

    /* 2) Direct system call: file descriptor 1 = the screen
     *    (standard output). 20 = number of bytes to write.   */
    write(1, "Hello from write()!\n", 20);

    return 0;
}
```

`printf()` is a *library function*: it runs entirely in user mode and formats the text. To actually put characters on the screen it must ask the kernel, using the `write()` system call. The program `strace` shows every system call another program makes:

```
$ gcc hello_syscall.c -o hello_syscall
$ strace ./hello_syscall
...
write(1, "Hello from printf!\n", 19)  = 19
write(1, "Hello from write()!\n", 20) = 20
exit_group(0)                         = ?
```

Both lines end up as the same `write` system call. The numbers are: file descriptor 1 (standard output, the screen), the bytes to write, their count, and, after the =, the value returned by the kernel (the number of bytes actually written).

Even a simple command makes many system calls. On our test machine `strace -c ls /` reported 76 system calls, most of them to open, read, map into memory and close files.

4.2 Example 2: how expensive is a system call?

```c
/* syscall_cost.c -- how expensive is a trip into the kernel?
 * We call a tiny ordinary function N times, then make a tiny
 * system call (getpid) N times, and compare the times.
 * Compile:  gcc -O2 syscall_cost.c -o syscall_cost
```

```
 * Run:      ./syscall_cost
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <time.h>
#include <unistd.h>
#include <sys/syscall.h>

#define N 1000000L

/* noinline: stop the compiler from optimising the call away */
__attribute__((noinline)) long plain_function(long x)
{
    return x + 1;
}

static double seconds(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec + t.tv_nsec / 1e9;
}

int main(void)
{
    volatile long sink = 0;   /* keeps results "used" */
    double t0, t1, t2;

    t0 = seconds();
    for (long i = 0; i < N; i++)
        sink += plain_function(i);          /* stays in user mode */
    t1 = seconds();
    for (long i = 0; i < N; i++)
        sink += syscall(SYS_getpid);        /* user -> kernel -> user */
    t2 = seconds();

    double f = (t1 - t0) / N * 1e9;         /* nanoseconds per call */
    double s = (t2 - t1) / N * 1e9;
    printf("ordinary function call : %6.1f ns\n", f);
    printf("system call (getpid)   : %6.1f ns\n", s);
    printf("system call is about %.0f times slower\n", s / f);
    return 0;
}
```

Sample output on our test machine (a Linux virtual machine):

```
$ gcc -O2 syscall_cost.c -o syscall_cost
$ ./syscall_cost
ordinary function call :    1.5 ns
system call (getpid)   :  116.1 ns
system call is about 76 times slower
```

Your numbers will differ: on a typical PC, a minimal system call costs roughly 50–200 ns, while a function call costs about 1–2 ns. The ratio is the lesson: crossing the boundary between user mode and kernel mode is *expensive*, because the CPU must save registers, switch modes, check the request and switch back. Keep this in mind for Section 5.4.

How much overhead is that?

With $t_{\text{func}} = 1.5$ ns and $t_{\text{sys}} = 116$ ns, making $N = 10^6$ system calls instead of function calls adds

$$N\,(t_{\text{sys}} - t_{\text{func}}) = 10^6 \times (116 - 1.5)\ \text{ns} \approx 1.1 \times 10^8\ \text{ns} \approx 0.11\ \text{s}.$$

5 Operating-System Structures

A general-purpose OS is a very large program; the Linux kernel alone has over 30 million lines of source code, most of it device drivers. How such a program is organised has a large effect on its speed, reliability and ability to evolve. We look at six approaches.

5.1 Simple structure: MS-DOS

MS-DOS (1981) was written to provide the most functionality in the least space on the original IBM PC. It was not divided into carefully separated modules.

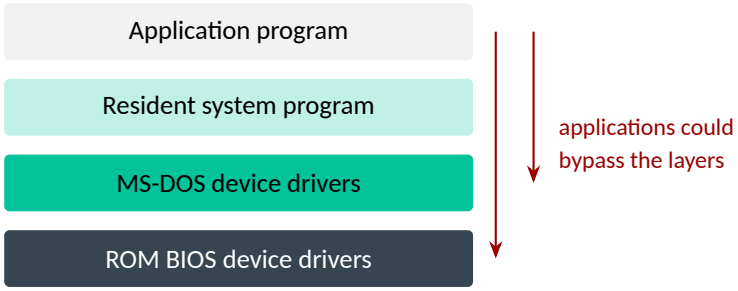


Figure 5: MS-DOS layer structure: programs could reach the hardware directly.

Application programs could bypass the OS and write directly to the display or the disk. The Intel 8088 processor of the IBM PC had *no* dual mode, so the OS had no way to stop them. As a result, one buggy program could crash the entire system. **Lesson:** protection requires both hardware support and an OS that uses it.

5.2 Monolithic kernel: UNIX and Linux

In a **monolithic** kernel, all OS services (process scheduling, memory management, file systems, networking, device drivers) form one large program that runs in kernel mode, in a single address space.

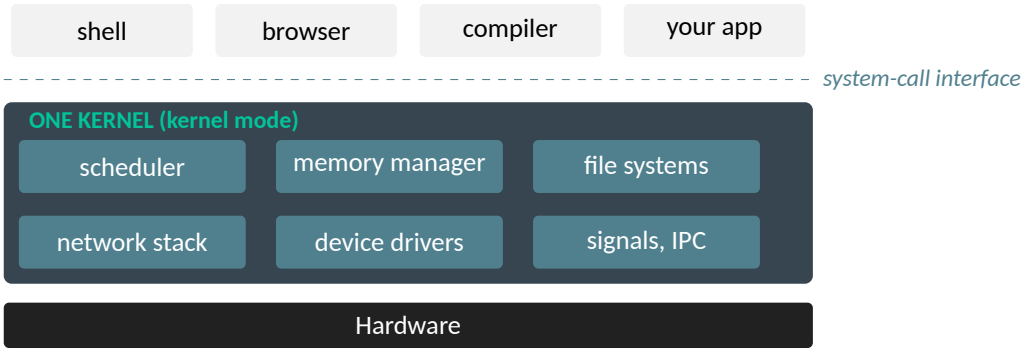


Figure 6: A monolithic kernel.

+ **Fast:** kernel components call each other directly, like ordinary function calls; a system call needs only two mode switches.
− **Hard to maintain:** millions of lines in one program.
− **Fragile:** a bug in any part, typically a driver, can crash the whole kernel (a *kernel panic* on Linux, a "blue screen" on Windows).

Examples: the original UNIX (1970s), Linux, traditional BSD.

5.3 Layered approach

The OS is divided into layers. Layer 0 is the hardware; the highest layer is the user interface. Each layer may use only the services of the layers below it.

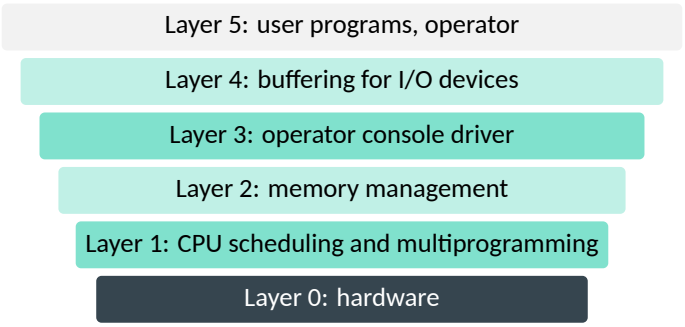


Figure 7: A layered OS, loosely based on Dijkstra's THE system (1968).

+ Easy to construct and debug: test layer 1 first, then layer 2 on top of it, and so on.
− It is hard to decide the order of the layers. Memory management needs the scheduler (to wait for a disk), but the scheduler needs memory (for its tables). Which goes first?
− Each request passes through many layers, each adding overhead.

Pure layering is rare in general-purpose systems today, but the idea survives inside every OS; the network stack, for example, is strictly layered.

5.4 Microkernel

The **microkernel** approach removes everything non-essential from the kernel and runs it as ordinary user-mode programs called *servers*. The microkernel itself provides only minimal process and memory manage-

ment and, above all, **message passing** between programs.

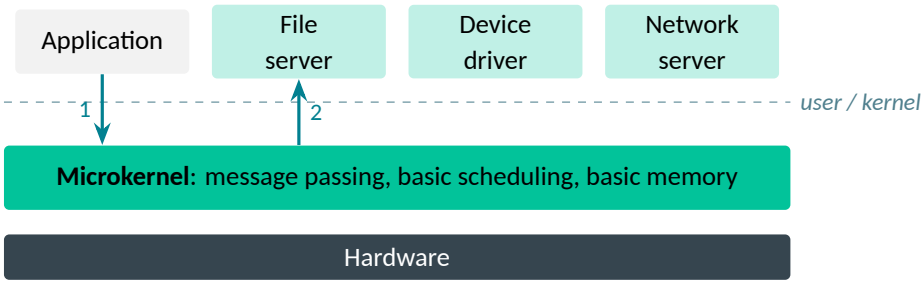


Figure 8: In a microkernel, an application's request to the file server travels as a message through the kernel.

+ **Reliable:** a crashed driver is just a crashed user program; the system can restart it.
+ **Secure and extensible:** new services are new user programs; the small kernel is easier to check. The seL4 microkernel has even been *mathematically proven* correct.
− **Slower:** every service request needs extra messages and mode switches.

Examples: Mach (Carnegie Mellon, 1980s), MINIX 3, QNX (used in cars and medical devices), L4 and seL4.

The price of a microkernel

Count the mode switches for one file read.

- Monolithic: application → kernel → application: 2 switches.
- Microkernel: application → kernel → file server → kernel → application: 4 switches, plus 2 message copies.

Take illustrative values: a mode switch costs $t_s = 0.1\ \mu$s, a message copy $t_c = 0.15\ \mu$s, and the useful work $t_w = 1\ \mu$s. Then

$$T_{\text{mono}} = 2t_s + t_w = 0.2 + 1.0 = 1.2\ \mu\text{s}, \qquad T_{\mu} = 4t_s + 2t_c + t_w = 0.4 + 0.3 + 1.0 = 1.7\ \mu\text{s},$$

so the microkernel is $T_{\mu}/T_{\text{mono}} \approx 1.42$ times, i.e. about 42%, slower for this operation. For long operations (large t_w) the ratio approaches 1 and the overhead hardly matters:

$$\lim_{t_w \to \infty} \frac{4t_s + 2t_c + t_w}{2t_s + t_w} = 1.$$

5.5 Modular kernel: loadable kernel modules

Most modern systems use a **modular** design: a core kernel plus **loadable kernel modules** that can be added or removed while the system is running. When you plug in a USB device, Linux can load the right driver module automatically.

+ Like a monolithic kernel, modules run in kernel mode, so calls between them are fast.
+ Like a microkernel, the system is flexible: only the modules that are needed are loaded.
− A buggy module still runs in kernel mode and can still crash the system.

Examples: Linux, Solaris, the modern BSDs.

Example 3: a minimal kernel module. The listing below is the standard "hello world" Linux kernel module. It has no `main()`; instead, the kernel calls `hello_init()` when the module is loaded and `hello_exit()` when it is removed. Its messages go to the kernel log, not to the screen.

```c
/* hello_module.c -- a minimal Linux loadable kernel module (LKM).
 * Build:   make                 (needs the kernel headers package)
 * Load:    sudo insmod hello_module.ko
 * Check:   lsmod | grep hello    and    sudo dmesg | tail -2
 * Unload:  sudo rmmod hello_module
 * Try this only in a virtual machine you are allowed to break!
 */
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("MCMP117");
MODULE_DESCRIPTION("Hello-world kernel module");

static int __init hello_init(void)      /* runs at insmod */
{
    pr_info("hello_module: loaded into the kernel\n");
    return 0;                           /* 0 = success */
}

static void __exit hello_exit(void)     /* runs at rmmod */
{
    pr_info("hello_module: removed from the kernel\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

```
$ make                          # needs the kernel-headers package
$ sudo insmod hello_module.ko   # load
$ lsmod | grep hello            # is it there?
$ sudo dmesg | tail -1          # read the kernel log
hello_module: loaded into the kernel
$ sudo rmmod hello_module       # unload
```

Safety first

Code in a kernel module runs in kernel mode: a mistake can freeze the whole computer. Only try Example 3 inside a virtual machine. It is shown here so that you can *see* what "loadable module" means; you will not need to write one.

5.6 Hybrid systems

Real operating systems rarely follow one pure design.

- **Windows (NT family)** was designed with microkernel ideas (separate subsystems), but for speed most services, including graphics, run in kernel mode. Drivers are loadable, like modules.
- **macOS and iOS** use the *XNU* kernel, which combines the Mach microkernel with parts of BSD UNIX, compiled into a single kernel, plus loadable kernel extensions.
- **Android** uses a modified Linux kernel (monolithic plus modules), with a hardware abstraction layer, native libraries and the Android runtime for apps on top.

The Tanenbaum–Torvalds debate (1992)

In January 1992, Andrew Tanenbaum, author of the MINIX microkernel, posted a message titled "LINUX is obsolete", arguing that a monolithic kernel was "a giant step back into the 1970s". Linus Torvalds replied that Linux worked and was fast. More than thirty years later, both designs are in daily use: Linux on servers and phones, microkernels in cars, aircraft and other safety-critical devices.

5.7 Comparison

Structure	In kernel mode	Speed	Reliability	Example
Simple	everything (no protection)	high	very low	MS-DOS
Monolithic	all OS services	high	medium	UNIX, Linux
Layered	all layers	low	high	THE (1968)
Microkernel	only the tiny kernel	lower	high	MINIX 3, QNX, seL4
Modular	core + loaded modules	high	medium	Linux, Solaris
Hybrid	varies	high	medium–high	Windows, macOS, Android

The recurring trade-off: every boundary you add (a layer, a message, a mode switch) buys reliability and costs speed.

6 Summary

Week 2 in six sentences

1. The OS manages the hardware and offers programs a simpler, safer machine; its always-resident core is the kernel.
2. It can be seen top-down as an extended machine (abstraction) or bottom-up as a resource manager (time and space multiplexing).
3. Its three design goals are convenience, efficiency and the ability to evolve, weighted differently on PCs, servers, phones and embedded systems.
4. A computer system has four components: hardware, OS, application programs and users; devices signal the CPU with interrupts.
5. Dual-mode operation protects the system; a system call switches to kernel mode and costs far more than a function call.

6. OS structures (simple, monolithic, layered, microkernel, modular, hybrid) are different compromises between speed and reliability.

7 Glossary

Abstraction
A simplified view that hides details, e.g. a *file* instead of disk sectors.

Buffering
Collecting data into large blocks to reduce the number of expensive operations.

Device driver
OS code that controls one type of device controller.

Dual-mode operation
Hardware support for two CPU modes, user and kernel, selected by a mode bit.

Interrupt
A signal from hardware that makes the CPU stop, run an OS handler, and then resume.

Interrupt vector
A table of handler addresses, indexed by interrupt number.

Kernel
The part of the OS that is always in memory and runs in kernel mode.

Loadable kernel module
Code that can be added to or removed from a running kernel.

Microkernel
A minimal kernel; most OS services run as user-mode server programs that communicate by messages.

Monolithic kernel
A kernel in which all OS services run as one program in kernel mode.

Multiplexing
Sharing a resource in time (taking turns) or in space (dividing it).

Privileged instruction
An instruction that the CPU executes only in kernel mode.

System call
A request from a program to the kernel; it switches the CPU to kernel mode.

Trap
A software-generated interrupt, caused by a system call or an error.

8 Try It Yourself

Look inside your own operating system

Linux (lab PCs, WSL, or a virtual machine):

```
$ uname -r          # which kernel version?
$ lsmod | head      # which modules are loaded?
$ strace -c ls      # count the system calls made by ls
```

Windows: run `winver` (Windows build) and `driverquery` (installed drivers) in a Command Prompt. In Task Manager → *Performance* → CPU, right-click the graph and choose *Show kernel times*.

1. How many kernel modules are loaded on your machine?
2. Which system call does `ls` use most often?
3. Find a program that spends a lot of time in kernel mode. Why might that be?

9 Review Questions

Conceptual

1. Give the three textbook definitions of an operating system. Which aspect does each emphasise?
2. Explain the difference between the "extended machine" and "resource manager" views. Give an example of each.
3. Give one example of time multiplexing and one of space multiplexing.
4. What is the difference between the kernel and a system program? Is the shell part of the kernel?
5. Name the three design goals of an OS. Which one matters most for (a) a smartphone, (b) a web server, (c) a car's brake controller?
6. Describe, step by step, what happens when a disk controller raises an interrupt.
7. Why must protection be supported by the hardware? What would happen on a CPU without dual mode?
8. What is the difference between `printf()` and `write()`?
9. Compare monolithic and microkernel designs: one advantage and one disadvantage of each.
10. Why is a modular kernel described as "the best of both worlds"? What is its remaining weakness?

Numerical

11. A system call costs 0.2 μs. A program reads a 1 MB file (a) one byte at a time, (b) in blocks of 4096 bytes. Compute the system-call overhead in each case. (Take 1 MB $= 2^{20}$ bytes.)
12. A device is polled 500 times per second; each check takes 2 μs. What fraction of the CPU is spent polling? How many CPU-seconds per day is that?

13. Using the scale "register access $= 1$ s", how long would an access to RAM (80 ns) and to an SSD ($100\ \mu$s) take? Take the register access time as 0.3 ns.

14. In the microkernel example, the useful work grows to $t_w = 10\ \mu$s. Compute the slow-down T_μ / T_{mono}. What does this tell you?

Answers to the numerical questions

11. (a) 2^{20} calls $\times\ 0.2\ \mu$s ≈ 0.21 s. (b) $2^{20}/2^{12} = 256$ calls $\times\ 0.2\ \mu$s $\approx 51\ \mu$s, i.e. 4096 times less.

12. $500 \times 2 \times 10^{-6} = 10^{-3} = 0.1\%$. Per day: $0.001 \times 86\,400$ s ≈ 86 CPU-seconds.

13. Scale factor $1/(0.3\text{ ns}) \approx 3.33 \times 10^9$. RAM: $80 \times 10^{-9} \times 3.33 \times 10^9 \approx 267$ s ≈ 4.4 min. SSD: $10^{-4} \times 3.33 \times 10^9 \approx 3.3 \times 10^5$ s ≈ 3.9 days.

14. $T_{\text{mono}} = 0.2 + 10 = 10.2\ \mu$s, $T_\mu = 0.4 + 0.3 + 10 = 10.7\ \mu$s; ratio ≈ 1.05, only 5% slower. The microkernel overhead matters for short, frequent operations, not for long ones.

10 Looking Ahead: Week 3

- The services an operating system provides to users and programs.
- System calls in detail: types, parameter passing, the system-call table.
- The process concept: a program in execution, and its memory layout.
- Process states: new, ready, running, waiting, terminated; the process control block.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley, Chapters 1–2.
- W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed., Pearson, Chapter 2.
- A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed., Pearson, Sections 1.1 and 1.7.
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, Chapter 6, "Limited Direct Execution": `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-mechanisms.pdf`